Broadband Networks 1 - Practical Project

Simulating a 5G Radio Access Network with OMNeT++ and Simu5G

A complete, from-zero walkthrough: what each tool is, how a network simulator works, and a step-by-step explanation of the three tasks - ALOHA, a custom wired-LAN example, and a simplified 5G RAN - together with the real results produced on the machine.

Component	Version	Role in one line
OMNeT++	6.1.0	The simulation engine and IDE (the foundation).
INET Framework	4.5.4	A library of ready-made Internet/network protocol models.
Simu5G	1.3.0	A 5G New Radio library built on top of INET.

How to read this document. Every technical word is explained the first time it appears, and all of them are collected again in the Glossary at the end. You do not need any prior knowledge of simulation or 5G to follow it.

1. The big picture: what are we actually doing?

The goal of this project is to **model a small 5G mobile network inside a computer program** and measure how it behaves - how much data gets through, how long it takes, how reliable it is - without owning a single antenna or phone. This is called **network simulation**.

1.1 Why simulate instead of using real equipment?

- **Cost.** A real 5G base station and licensed spectrum cost a fortune. A simulation is free.
- **Control.** You can place users anywhere, change the radio power, add 100 phones, or break a link, all by editing a text file.
- **Repeatability.** The exact same experiment can be re-run and will give the same numbers, so results can be compared fairly.
- **Safety and speed.** You can test ideas that would be illegal or dangerous on real spectrum, and run a 20-second experiment in a fraction of a second.

1.2 What is a discrete-event simulator?

All three tools here are built around one idea: **discrete-event simulation (DES)**. Instead of advancing time smoothly like a clock, the simulator jumps from one **event** to the next. An event is something that happens at a precise instant: a packet is sent, a packet arrives, a timer fires, a collision starts.

The simulator keeps a list of future events ordered by time, called the **event queue**. It repeatedly takes the earliest event, jumps its internal **simulation time** straight to that moment, runs the small piece of code attached to it (which may schedule new future events), and moves on. Between two events nothing happens, so the simulator simply skips that empty time.

> **Analogy.** Imagine reading a train timetable instead of watching the clock all day. You do not care about the minutes when nothing happens; you jump from one departure to the next. 'Simulation time' is the timetable clock - it can represent 20 seconds of network activity while the program runs for a few milliseconds of real time, or the other way around for heavy scenarios.

1.3 The three programs and how they stack

The three tools are not competitors; they sit **on top of each other**, each adding a layer of capability. You always run them together.

OMNeT++ is the **engine**. On its own it knows nothing about Ethernet, IP or 5G. It only provides the generic machinery of simulation: how to describe a network out of building blocks (modules), how those blocks exchange **messages**, how to schedule events, how to record results, and a graphical interface to watch it all. Think of it as a game engine with no game loaded yet.

INET is a huge **library of network models** written for OMNeT++. It provides ready-made modules for real protocols and devices: Ethernet cards, hubs, switches, IP routers, TCP, UDP, WiFi, hosts, applications, and so on. Without INET you would have to build a 'host' or a 'router' from scratch. Simu5G also needs INET, because a 5G phone still uses IP and UDP above the radio.

Simu5G is a **5G library** built on top of INET. It adds everything specific to 5G New Radio: the base station (gNB), the user terminal (UE), the radio channel, the scheduler that shares the air among users, and the 5G core network functions. It reuses INET for the ordinary Internet parts and focuses on the radio.

Layer	What it provides
Your scenario	The .ini and .ned files you write (e.g. '1 gNB + 5 phones').
Simu5G 1.3.0	5G New Radio: gNB, UE, radio channel, scheduler, 5G core.
INET 4.5.4	Ethernet, IP, TCP/UDP, WiFi, hosts, routers, applications.
OMNeT++ 6.1.0	Simulation engine: modules, messages, events, results, GUI.
Linux / C++	The operating system and compiler that run it all.

Each task in this project exercises a different layer: Task 1 uses **OMNeT++ alone**, Task 2 adds **INET**, and Task 3 adds **Simu5G** on top.

2. Anatomy of an OMNeT++ project

Before looking at the tasks, it helps to know the handful of concepts that every OMNeT++ simulation is made of. They are the same whether the network is an old ALOHA radio or a modern 5G cell.

2.1 Modules and the network

Everything in OMNeT++ is a **module**. A **simple module** is a box with behaviour, written in C++ (for example, an application that generates packets). A **compound module** is a box made of other modules wired together (for example, a 'host' that contains an application, a protocol stack and a network card). The top-level compound module is called the **network** - it is the whole scenario.

2.2 Two kinds of files: .ned and .ini

A simulation is described by two kinds of text files, and understanding the split is the single most useful thing to learn:

- **.ned files (the structure).** NED stands for Network Description. A .ned file says *what modules exist and how they are connected* - the topology. It is like the wiring diagram.
- **.ini files (the settings).** The omnetpp.ini file says *what values the parameters take* for a given run - how many hosts, how fast the link, how often a packet is sent, how long to simulate. It is like the knobs and dials on that diagram.

The same .ned topology can be run with many different .ini settings. That separation is exactly what Task 2 of the project asks you to explore.

A tiny NED example (two nodes linked by a cable) looks like this:

```
network Tiny
{
    submodules:
        a: StandardHost;
        b: StandardHost;
    connections:
        a.ethg++ <--> Eth100M <--> b.ethg++;
}
```

And a matching .ini fragment that sets parameters:

```
[General]
network = Tiny
sim-time-limit = 10s
**.a.numApps = 1
**.a.app[0].typename = "UdpBasicApp"
```

2.3 The vocabulary of an .ini file

- **[General]** is the section with settings shared by every run.
- **[Config Name]** defines a named experiment. You pick one with the `-c` option.
- **extends = Other** lets one config inherit everything from another and change only a few lines.
- **Wildcards.** `*` matches one name level and `**` matches any depth, so `**.host[*].iaTime` sets a parameter on every host at once.

2.4 Messages, events and time

Modules never call each other directly; they communicate by sending **messages** (a packet is a message). Sending a message to another module, or to itself as a timer, creates a future **event**. The engine processes events in time order. This is the heartbeat of the whole system.

2.5 Running a simulation: Qtenv vs Cmdenv

The same compiled model can be launched in two **environments**:

- **Qtenv** is the graphical interface. You see the network, watch packets move, inspect any module, and step through events. It is the best way to understand and debug a model - every screenshot in this document is Qtenv.
- **Cmdenv** is the command-line interface. No graphics, just runs as fast as possible and writes the results. It is used for real experiments and for collecting numbers in batches.

2.6 Where the results go

When a run finishes it writes files into a **results/** folder:

- **Scalar file (.sca).** One value per statistic for the whole run - for example the average delay, the total number of collisions, the channel utilisation. These are the headline numbers.
- **Vector file (.vec).** A full time series - every recorded sample with its timestamp - for drawing curves over time.

A statistic named like `cbrReceivedThroughput:mean` means 'the mean value of the received-throughput statistic'. The `:mean`, `:sum`, `:count` suffixes tell you how the many samples were reduced to one number.

2.7 How a .ned file is written, line by line

A .ned file answers one question: **what devices exist and how are they wired?** Below is the complete custom topology used in Task 2 - a shared hub with six hosts - followed by what every line does.

```
package inet.examples.ethernet.lans;
import inet.node.ethernet.EthernetHub;
import inet.node.ethernet.EthernetHost;

network BigHubLAN
{
    submodules:
        hostA: EthernetHost { eth.duplexMode = false; }
        ...
        hostF: EthernetHost { eth.duplexMode = false; }
        hub:   EthernetHub  { gates: ethg[6]; }
    connections:
        hub.ethg[0] <--> { datarate = 100Mbps; } <--> hostA.ethg;
        ...
}
```

NED line	What it does
`package ... .lans;`	The folder/namespace this network belongs to. It must match the example's package so the tools can locate the file.
`import ... EthernetHub;`	Pulls in a ready-made module type from INET, so we can use 'EthernetHub' by name below.
`network BigHubLAN {`	Starts the definition of a network - the top-level box that is the whole scenario.
`submodules:`	Begins the list of devices that live inside this network.
`hostA: EthernetHost { }`	Creates one device named hostA of type EthernetHost. The braces set that device's parameters.
`eth.duplexMode = false;`	Puts this host's Ethernet card in half-duplex - which is exactly what lets a shared hub collide.
`hub: EthernetHub { gates: ethg[6]; }`	Creates the hub and gives it six ports (ethg[6]) so six hosts can plug in.
`connections:`	Begins the list of cables between devices.
`hub.ethg[0] <--> {...} <--> hostA.ethg;`	Wires hub port 0 to hostA. '<-->' is a two-way link; the braces set the cable's data rate (100 Mbps) and delay.

2.8 How an .ini file is written, line by line

An .ini file answers the other question: **what values do the parameters take for this run?** The parameter names use the same dotted paths as the modules in the .ned, with wildcards to set many at once: * matches one name level, ** matches any depth, and [*] matches every index of an array. Below are the key lines of the Task 3 config.

```
[Config TP-RAN-DL]
network = simu5g.simulations.NR.networks.SingleCell_Standalone
sim-time-limit = 20s
*.numUe = 5
*.ue[*].macCellId = 0
*.ue[*].nrMasterId = 1
*.ue[*].mobility.speed = uniform(1mps, 5mps)
*.server.numApps = 5
*.server.app[*].typename = "CbrSender"
*.server.app[*].packetSize = 1000B
*.server.app[*].sampling_time = 0.01s
*.server.app[*].destAddress = "ue[" + string(ancestorIndex(0)) + "]"
```

INI line	What it does
`[Config TP-RAN-DL]`	Defines a named experiment. You pick it on the command line with '-c TP-RAN-DL'.
`network = ...Standalone`	Chooses which .ned network to run - here the 5G standalone single-cell topology.
`sim-time-limit = 20s`	Stops the run after 20 seconds of simulated time.
`*.numUe = 5`	A network-level parameter: create five UEs (phones).
`*.ue[*].macCellId = 0`	Applies to every UE (ue[*] = all indices). Together with the other id lines it attaches each phone to our single base station - the 'which antenna serves me' setting.
`*.ue[*].mobility.speed = uniform(1mps,5mps)`	Each UE gets a random speed between 1 and 5 m/s; parameters can be random expressions, not just fixed numbers.
`*.server.numApps = 5`	The server runs five applications - one data stream per phone.
`*.server.app[*].typename = "CbrSender"`	Sets the type of each app: a Constant-Bit-Rate sender.
`packetSize = 1000B / sampling_time = 0.01s`	Each stream sends a 1000-byte packet every 10 ms - a steady, predictable load.
`destAddress = "ue["+string(ancestorIndex(0))+"]"`	App number i sends to ue[i]: ancestorIndex(0) is the app's own index, so each stream is routed to a different phone.

The key idea. The .ned never changes between experiments - it is the fixed wiring. You run completely different scenarios just by editing parameter values in the .ini, which is why the two files are kept separate.

3. Task 1 - ALOHA: validating the environment

Objective. Run the classic ALOHA example that ships with OMNeT++, confirm the installation works, understand the structure of a project, and read the basic results (collisions, success rate). This task uses **OMNeT++ alone** - no INET, no Simu5G.

3.1 What is ALOHA?

ALOHA is one of the oldest ways to let many senders share a single radio channel, invented at the University of Hawaii in 1971. The rule is deliberately naive: **whenever a station has a packet, it just transmits.** There is no listening, no coordination. If two stations happen to transmit at overlapping times, their signals **collide** and both packets are destroyed and must be resent later.

Because collisions waste the channel, ALOHA can never use the medium fully. Theory shows that **pure ALOHA** can carry at most **1/(2e) = about 18.4%** of the channel's raw capacity, and only at one particular, moderate level of traffic. Push more traffic in and collisions multiply; send too little and the channel sits idle. Task 1 is about *seeing that curve* with your own simulation.

3.2 The example and what you see

The model is 20 wireless hosts scattered on a map, all talking to one central **server** (the receiver). Each host waits a random time, drawn from an exponential distribution with mean **iaTime** (the **inter-arrival time**), then sends a fixed-size packet. The server reports how much of the channel was used and how much time was lost to collisions.

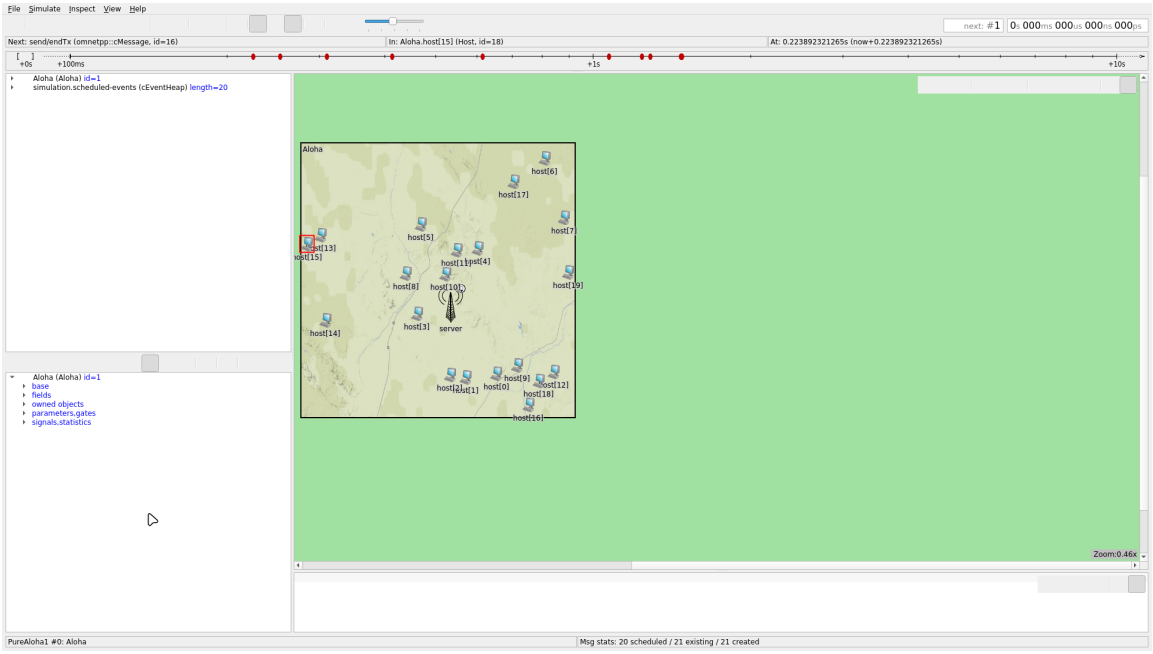


Figure 1. The ALOHA model in the Qtenv GUI. The map shows the 20 hosts; the horizontal strip near the top is the event timeline, where each red mark is a collision event. The left panels let you inspect any module and its statistics in real time.

3.3 The three load scenarios

The omnetpp.ini defines three ready-made configs that differ only in how often each host transmits - that is, in the offered load:

- **PureAloha1 - overloaded.** iaTime = exponential(2s): hosts transmit very often, so collisions are frequent.
- **PureAloha2 - near optimal.** iaTime = exponential(6s): traffic is tuned close to the point of maximum throughput.
- **PureAloha3 - light traffic.** iaTime = exponential(30s): hosts rarely transmit, so collisions are rare but the channel is mostly idle.

Each scenario was run for 5000 seconds of simulation time. The real numbers produced on the machine:

Scenario	Offered load	Channel utilisation	Frames received	Time lost to collisions
PureAloha1	High (mean 2s)	14.6 %	7 349	2 376 s
PureAloha2	Tuned (mean 6s)	17.3 %	8 742	531 s
PureAloha3	Low (mean 30s)	5.7 %	2 851	29 s

3.4 Reading the results

These three rows **are** the ALOHA throughput curve, measured:

- **The optimal case wins.** PureAloha2 reaches 17.3% channel utilisation - essentially the theoretical pure-ALOHA ceiling of 18.4% (1/(2e)). This confirms both the theory and that the installation computes correctly.
- **Overload collapses throughput.** PureAloha1 offers more traffic yet delivers *less* (14.6%), because 2 376 seconds - almost half of the useful air-time - were burned on collisions.
- **Underload wastes the channel.** PureAloha3 almost never collides (only 29 s lost) but the channel is idle most of the time, so utilisation is just 5.7%.

What Task 1 proves. The environment is correctly installed and produces physically meaningful results: a textbook phenomenon (the 1/(2e) limit and the collision trade-off) emerged from the simulation without any tuning on our part.

4. Task 2 - A custom wired example (OMNeT++ + INET)

Objective. Pick and run a different example, then **customise** its configuration (.ini) and topology (.ned) to understand how those files drive a simulation. This task adds the **INET** library.

4.1 The example chosen and why

I chose INET's **Ethernet LAN** example (`examples/ethernet/lans`). It is the wired cousin of ALOHA and makes the customisation lesson very visual, because Ethernet faces the *same* collision problem as ALOHA but solves it with better technology. It lets us compare two devices side by side:

- **Hub.** An old, dumb device that simply copies every incoming signal to all its ports. All hosts therefore share one **collision domain** and must take turns, exactly like ALOHA. Ethernet's rule here is **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection): listen before sending, and if a collision is detected, stop and retry.
- **Switch.** A smart device that reads each frame's destination address and forwards it only to the correct port. Each link becomes its own **full-duplex** collision domain, so collisions essentially disappear.

4.2 The customisation I made

(a) Customising the .ini. I wrote a new config file that applies the *same heavy load* (a new frame every 2 ms on average) to both a hub network and a switched network, so the only difference is the device. This is the essence of editing an .ini: changing parameters to set up a controlled experiment.

```
[General]
sim-time-limit = 20s
**.cli.sendInterval = exponential(2ms)
**.cli.destAddress = "hostA"

[Config TP-Hub]
network = HubLAN

[Config TP-Switch]
network = SwitchedLAN
**.switch*.bridging.typename = "MacRelayUnit"
```

(b) Customising the .ned. To show topology editing, I wrote my own network describing a *larger* shared hub with **six** hosts instead of four. Each host is a module, each <--> line is a cable with a delay and a 100 Mbps data rate:

```
network BigHubLAN
{
    submodules:
        hostA: EthernetHost { eth.duplexMode = false; }
        hostB: EthernetHost { eth.duplexMode = false; }
        hostC: EthernetHost { eth.duplexMode = false; }
        hostD: EthernetHost { eth.duplexMode = false; }
        hostE: EthernetHost { eth.duplexMode = false; }
        hostF: EthernetHost { eth.duplexMode = false; }
        hub:   EthernetHub { gates: ethg[6]; }
    connections:
        hub.ethg[0] <--> { datarate = 100Mbps; } <--> hostA.ethg;
        hub.ethg[1] <--> { datarate = 100Mbps; } <--> hostB.ethg;
        hub.ethg[2] <--> { datarate = 100Mbps; } <--> hostC.ethg;
        hub.ethg[3] <--> { datarate = 100Mbps; } <--> hostD.ethg;
        hub.ethg[4] <--> { datarate = 100Mbps; } <--> hostE.ethg;
        hub.ethg[5] <--> { datarate = 100Mbps; } <--> hostF.ethg;
}
```

`duplexMode = false` forces half-duplex, which is what makes the shared hub able to collide. Setting it `true` (as the switch does) is what removes collisions.

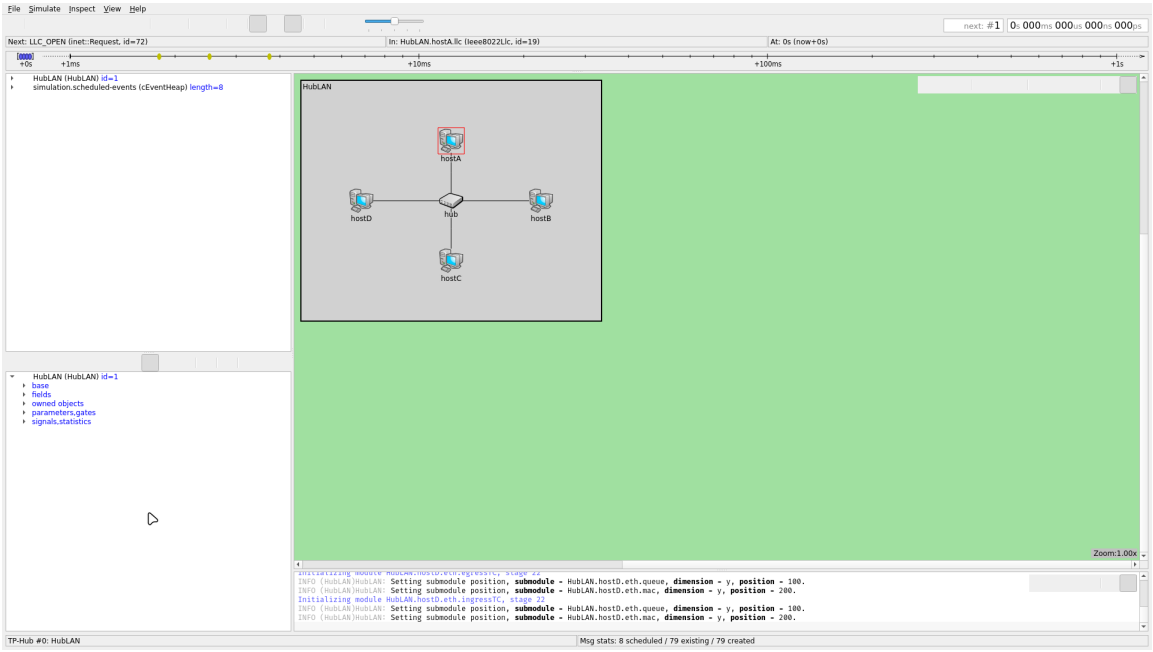


Figure 2. The four-host HubLAN in Qtenv: hostA, hostB, hostC and hostD all wired to a single central hub. Because a hub shares one collision domain, these four nodes behave like ALOHA stations competing for one channel.

4.3 Results

All three configs ran for 20 s under the same heavy load. The engine records a `collisions` counter and a 'rx channel collision (%)' figure on each host's Ethernet card:

Config	Topology	Total collisions	Channel time lost	Frames rx (hostA)
TP-Hub	4 hosts, hub	172 647	23.9 %	29 957
TP-Switch	4 hosts, switch	0	0 %	30 148
TP-Hub-Big	6 hosts, hub (my .ned)	500 245	45.4 %	47 954

- **The switch eliminates collisions.** Same traffic, same load - but zero collisions, because each host owns its link. This is why modern LANs use switches, not hubs.
- **More contenders means more collisions.** Going from four to six hosts on a shared hub nearly tripled the collisions and pushed lost air-time from 24% to 45%. This is the same lesson as ALOHA's overload case, now in the wired world.

What Task 2 proves. By editing only text files - an .ini to change the load and the device, a .ned to change the topology - we set up and answered a real engineering question (hub vs switch, and the cost of more users on a shared medium) without writing any C++.

5. Task 3 - A simplified 5G RAN with Simu5G

Objective. Build a basic 5G topology with at least one base station (gNB) and a set of user terminals (UEs), run a user-plane simulation, and observe the radio-access performance metrics. This task adds the **Simu5G** library on top of INET.

5.1 A five-minute 5G vocabulary

- **RAN (Radio Access Network).** The radio side of a mobile network: the antennas and the phones, plus the radio protocols between them. This is what we simulate.
- **gNB (gNodeB).** A **5G base station** - the antenna site your phone connects to. The 'g' is for the next (g)eneration; in 4G it was called eNodeB.
- **UE (User Equipment).** Any 5G terminal: a phone, a tablet, a car modem. In the simulation each UE is a module that attaches to a gNB.
- **Standalone (SA).** A full 5G network where the 5G radio is paired with a brand-new **5G core**. (The alternative, Non-Standalone, leans on an existing 4G core.) We use SA.
- **User plane vs control plane.** The **user plane** is the actual user data (your video, your call). The control plane is the signalling that sets things up. Task 3 measures the user plane.
- **UPF (User Plane Function).** The box in the 5G core that routes user data between the Internet and the right gNB. In the figure the path is server -> router -> UPF -> gNB -> UE.
- **CQI (Channel Quality Indicator).** A number from 0 to 15 the UE reports to say how good its radio link is right now. Higher means a cleaner channel, which lets the gNB send faster.
- **Resource blocks.** The air is divided into small time-frequency tiles; the gNB's **scheduler** hands these tiles out to UEs every millisecond. More tiles to a UE means more speed for that UE.

5.2 The scenario I built

Starting from Simu5G's `SingleCell_Standalone` network, I created a custom config with **one gNB and five mobile UEs**. A server on the Internet sends each UE a steady **CBR (Constant Bit Rate)** downlink stream - a 1000-byte packet every 10 ms, which is a simple, predictable user-plane load. The key parts of the config:

```
[Config TP-RAN-DL]
network = simu5g.simulations.NR.networks.SingleCell_Standalone
sim-time-limit = 20s

*.numUe = 5
*.ue[*].macCellId = 0
*.ue[*].masterId = 0
*.ue[*].nrMacCellId = 1
*.ue[*].nrMasterId = 1

*.ue[*].mobility.typename = "LinearMobility"
*.ue[*].mobility.speed = uniform(1mps, 5mps)

*.server.numApps = 5
*.server.app[*].typename = "CbrSender"
*.server.app[*].packetSize = 1000B
*.server.app[*].sampling_time = 0.01s
*.ue[*].app[*].typename = "CbrReceiver"
```

- **numUe = 5** creates the set of five terminals the objective asks for.
- **macCellId / masterId / nrMacCellId / nrMasterId** attach every UE to our single gNB (its cell identifiers). This is the line that says 'this phone is served by that antenna'.
- **LinearMobility** makes the UEs move slowly (1-5 m/s, i.e. walking pace), so the radio link varies a little, as it would in reality.
- **CbrSender / CbrReceiver** are the user-plane application: the server sends, each UE receives.

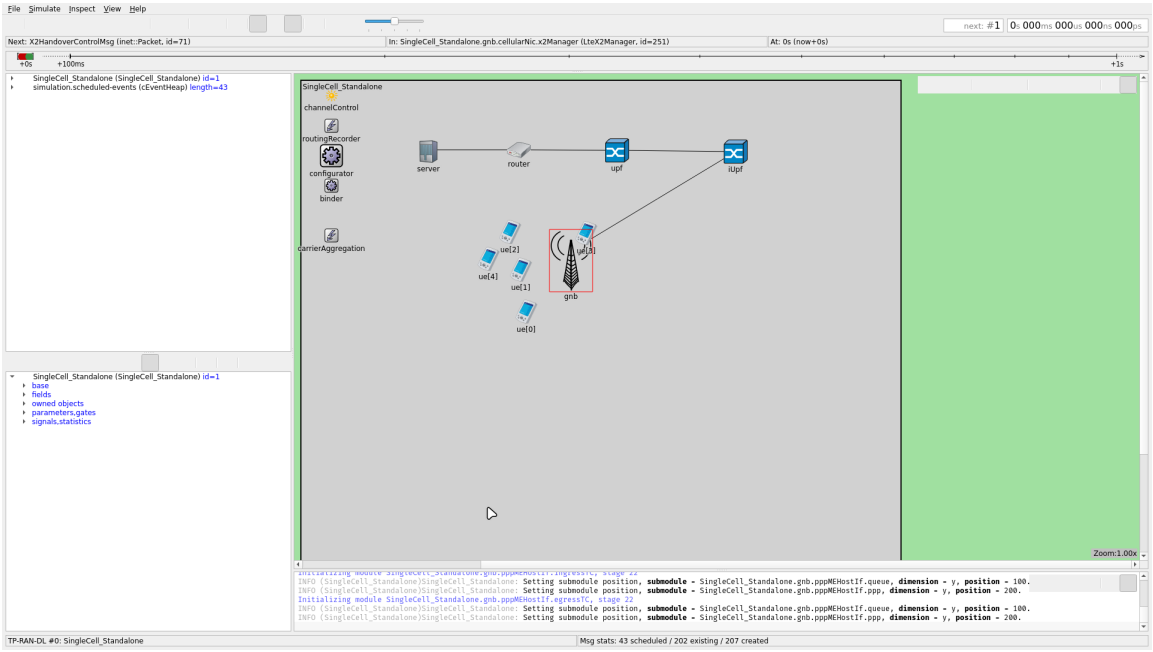


Figure 3. The 5G Standalone topology in Qtenv. On the wired side: the Internet 'server', a 'router', and the 5G core's 'upf'. On the radio side: the 'gnb' base station (antenna icon) serving the five terminals ue[0]..ue[4]. Data flows server -> router -> upf -> gnb -> ue.

5.3 Results - the radio-access metrics

After 20 s, Simu5G records a rich set of per-UE and per-cell statistics. The headline user-plane metrics for each terminal:

Terminal	App throughput	End-to-end delay	Packet loss	Avg DL CQI (0-15)
ue[0]	~100 kB/s (0.80 Mb/s)	4.78 ms	0	14.7
ue[1]	~100 kB/s (0.80 Mb/s)	4.87 ms	0	15.0
ue[2]	~100 kB/s (0.80 Mb/s)	4.55 ms	0	14.4
ue[3]	~100 kB/s (0.80 Mb/s)	4.32 ms	0	15.0
ue[4]	~100 kB/s (0.80 Mb/s)	4.55 ms	0	14.9

And at the base station, the aggregate cell throughput:

Cell-level metric (at the gNB)	Value
MAC-layer downlink throughput	514 393 B/s (~4.1 Mb/s)
RLC-layer downlink throughput	512 401 B/s (~4.1 Mb/s)

5.4 Reading the results

- **Every UE is fully served.** Each terminal receives its full ~0.80 Mb/s stream with **zero packet loss**, so the cell comfortably carries all five users.
- **Latency is low and consistent.** End-to-end delay sits around 4-5 ms for every UE - exactly the kind of responsiveness 5G promises.
- **The channel is excellent.** Average CQI is near the maximum of 15, because the UEs are close to the gNB with little interference, so the scheduler can use the fastest transmission settings.
- **The numbers add up.** Five UEs at ~0.80 Mb/s each equal roughly 4.0 Mb/s, which matches the ~4.1 Mb/s measured at the gNB's MAC layer - the small extra is protocol overhead. The parts are consistent, a good sign the model behaves sensibly.

What Task 3 proves. A working 5G Standalone radio access network - core, base station and five moving users exchanging real user-plane traffic - was built and measured, producing the standard KPIs an operator cares about: throughput, latency, loss and radio quality.

6. A little more 5G context

Task 3 simulates the **user plane of a 5G Standalone network**. To place that in the bigger picture described in the course chapter:

6.1 Standalone vs Non-Standalone

When 5G was standardised, two deployment options dominated. **Option 3 (Non-Standalone, NSA)** keeps the existing 4G/LTE core and uses 5G radio only as an extra speed boost next to a 4G anchor - quick to deploy, but limited to faster mobile broadband. **Option 2 (Standalone, SA)** connects 5G radio to a new **5G Core**, unlocking the full feature set: a cloud-friendly Service-Based Architecture, ultra-reliable low-latency communication, and network slicing. Our simulation is Option 2.

6.2 Network slicing (why SA matters)

The flagship feature of the 5G core is **network slicing**: running several independent virtual networks on one shared physical infrastructure, each tuned for a different need - one slice for high-speed video, one for tiny ultra-reliable control messages, one for millions of simple sensors. The same antenna and the same core serve all of them, isolated from one another. This is the architectural reason operators moved to Standalone 5G.

7. How to reproduce everything

Each part can be re-run from a terminal. First load the environment once per shell:

```
cd ~/omnetpp-ws/omnetpp-6.1 && source ./setenv
cd ~/omnetpp-ws/inet4.5     && source ./setenv
```

Task 1 - ALOHA:

```
cd ~/omnetpp-ws/omnetpp-6.1/samples/aloha
./aloha -u Cmdenv -c PureAloha2 --sim-time-limit=5000s
# results in ./results/*.sca
```

Task 2 - Ethernet hub vs switch:

```
cd ~/omnetpp-ws/inet4.5/examples/ethernet/lans
opp_run -u Cmdenv -c TP-Hub    -l ~/omnetpp-ws/inet4.5/src/INET \
        -n ~/omnetpp-ws/inet4.5/src:~/omnetpp-ws/inet4.5/examples tp_lan.ini
# repeat with -c TP-Switch and -c TP-Hub-Big
```

Task 3 - 5G RAN:

```
cd ~/omnetpp-ws/Simu5G-1.3.0/simulations/NR/standalone
opp_run -u Cmdenv -c TP-RAN-DL \
  -l ~/omnetpp-ws/Simu5G-1.3.0/src/simu5g -l ~/omnetpp-ws/inet4.5/src/INET \
  -n ~/omnetpp-ws/Simu5G-1.3.0/src:~/omnetpp-ws/Simu5G-1.3.0/simulations:\
~/omnetpp-ws/inet4.5/src tp_ran.ini
```

Replace `-u Cmdenv` with `-u Qtenv` to open the graphical interface and watch any of them run.

8. Glossary

ALOHA. The 1971 random-access scheme where stations transmit whenever they wish; simple but limited to ~18.4% channel use because of collisions.

CBR (Constant Bit Rate). A traffic source that sends fixed-size packets at a fixed rate - a simple, predictable load used to measure a network.

Channel utilisation. The fraction of a channel's raw capacity actually carrying useful data (the rest is idle or lost to collisions).

Cmdenv. OMNeT++'s command-line run environment: no graphics, maximum speed, used for batch experiments.

Collision. Two transmissions overlapping on a shared medium, destroying both and forcing a retransmission.

Collision domain. A set of devices that share one medium and can collide with each other. A hub is one big collision domain; a switch gives each link its own.

CQI (Channel Quality Indicator). A 0-15 value a UE reports about its radio link quality; higher lets the base station transmit faster.

CSMA/CD. Ethernet's shared-medium rule: listen before sending and abort on a detected collision.

Discrete-event simulation. A method that advances time by jumping from one scheduled event to the next, rather than ticking continuously.

Event. Something that happens at a precise simulation instant (a send, an arrival, a timer).

gNB (gNodeB). A 5G base station - the antenna site a UE connects to.

Full-duplex / half-duplex. Full-duplex links send and receive at the same time (no collisions); half-duplex must take turns (can collide).

Hub. A dumb device that copies every signal to all ports, creating one shared collision domain.

INET. A large library of Internet/network protocol models (Ethernet, IP, TCP/UDP, WiFi, ...) for OMNeT++.

Inter-arrival time (iaTime). The average gap between a source's successive transmissions; small means heavy load.

.ini file. The configuration file setting parameter values and defining named experiments.

Jitter. The variation in packet delay.

Module. The basic building block of an OMNeT++ model; simple modules hold C++ behaviour, compound modules contain other modules.

.ned file. The Network Description file defining which modules exist and how they connect (the topology).

OMNeT++. The discrete-event simulation engine and IDE underneath everything here.

Qtenv. OMNeT++'s graphical run environment for watching and inspecting a simulation.

RAN (Radio Access Network). The radio side of a mobile network: base stations, terminals and their radio protocols.

Resource block. A small time-frequency tile of the radio spectrum that the gNB scheduler allocates to UEs.

Scalar / vector result. A scalar is one summary value per run; a vector is a full time series of samples.

Simu5G. A 5G New Radio simulation library built on top of INET and OMNeT++.

Standalone (SA) / Non-Standalone (NSA). SA pairs 5G radio with a new 5G core; NSA reuses an existing 4G core as an anchor.

Switch. A smart device that forwards each frame only to its destination port, removing collisions.

Throughput. The rate of useful data successfully delivered, in bits or bytes per second.

UE (User Equipment). A 5G terminal such as a phone, tablet or modem.

UPF (User Plane Function). The 5G-core element that routes user data between the Internet and the serving gNB.

User plane / control plane. The user plane carries actual user data; the control plane carries the signalling that manages connections.